Implementation and mesh assignments for staggered MP3 CCD and G0W0 AC

The three implementations use different momentum meshes for different parts of a contraction. Three-mesh MP3 evaluates one external amplitude configuration using several auxiliary MP2 configurations. Three-mesh CCD iterates a coupled set of amplitude configurations and averages several energy estimates. Staggered G0W0 uses two orbital meshes for its main self-energy calculation and a separate target-mesh calculation for its long-wavelength correction.

This document describes the current executable paths, including mesh construction, term assignments, and finite-size corrections. The final section compares them with the method outlines in `docs`.

Method	Main implementation	Method outline
MP3	`KMP3_3mesh` and the inherited `KMP3.kernel`	Staggered MP3, Sections 2–3
CCD	`KRCCD_3mesh` and its intermediates	Three-mesh CCD, Sections 0.1–0.5
G0W0 AC	`krgw_ac_stagger.py`	Staggered GW, Sections 1–2 and Appendix .2

The existing migration and validation notes give numerical comparisons and provenance. The November GW results contain numerical plots; the September GW note contains the method outline.

Mesh notation and momentum conservation

Let `n = (n1, n2, n3)` be the shape of one correlation mesh and `N = n1*n2*n3` its number of points. Let `U(n)` be the Gamma-centered mesh returned by `cell.make_kpts(n, with_gamma_point=True)`. All shifts below are **fractional reciprocal coordinates**; `cell.get_abs_kpts` converts them to Cartesian wavevectors. Points are equivalent modulo reciprocal lattice vectors.

For MP3 and CCD, mesh labels `0`, `1`, and `2` mean

```
K0 = anchor + U(n)
K1 = K0 + delta
K2 = K0 + 2*delta
```

They correspond to the outlines' labels `0`, `1/3`, and `2/3`. The same label can carry occupied or virtual orbitals: these are not three fixed orbital classes. A tuple `(mi, mj, ma, mb)` specifies the meshes of `(I, J, A, B)` in an amplitude `T_IJ^AB`, with `I,J` occupied and `A,B` virtual.

Momentum conservation fixes the fourth point:

```
kb = ki + kj - ka                  modulo reciprocal lattice vectors
mb = (mi + mj - ma) % 3
```

The actual point is obtained from `khelper.kconserv[ki, ka, kj]`. The shared `make_kpts_helper` builds this map from fractional coordinates, so it works with concatenated meshes whose ordering differs from a canonical Monkhorst–Pack grid. `kpts_base_idx[m]` maps local points on `Km` to global indices; `idx_base[m]` maps them back.

The transfer mesh between two orbital meshes is `U(n) + (ma-mi)*delta`; the common anchor cancels. Different mesh labels avoid zero transfer in the corresponding ERI channel. This removes direct sampling of the singular point, but a Madelung correction can still be needed for the quadrature error around that point.

How the orbital meshes are constructed

MP3 and CCD submesh mode

Both classes accept `flag_submesh=True`. If the reference SCF mesh has shape `R`, each component greater than one must be divisible by three. The code sets

```
n_alpha     = R_alpha/3 if R_alpha > 1 else 1
delta_alpha = 1/R_alpha if R_alpha > 1 else 0
```

The anchor is the supplied SCF k-point of smallest Cartesian norm. The code constructs `K0`, `K1`, and `K2` and locates them in the original reference mesh after wrapping coordinates. It reuses the reference orbitals, orbital energies, and density-fitting object. The three selected meshes must be disjoint; a Gamma-only reference cannot supply them.

For a reference `(6,6,6)`, each selected mesh has shape `(2,2,2)` and the shifts are `(1/6,1/6,1/6)` and `(1/3,1/3,1/3)`. Only 24 of the 216 reference points belong to these three meshes. The selection shifts all sampled axes together; it does not select all 27 combinations of independent axis shifts. The original reference mesh remains the global orbital container.

MP3 and CCD band extension mode

`flag_submesh=False` is the default. The SCF density is held fixed while `get_bands` computes orbitals and energies on the concatenated three meshes. For both methods this temporarily uses FFTDF; the KRHF branch also uses `exxdiv='vcut_sph'`. A fresh GDF object is then built on the union for the correlation integrals. Band extension requires a three-dimensional cell.

The constructors differ in their treatment of mesh shape and one-point axes:

Constructor	Base mesh `K0`	Fractional shift `delta`
MP3 without `nks_mp`	The original `mf.kpts`, with its original offset	`1/(3*n_alpha)` on axes with `n_alpha > 1`; zero on the other axes
MP3 with `nks_mp=n`	A new Gamma-centered `U(n)`	The same rule, now using `nks_mp`
MP3 with `n=(1,1,1)`	As selected above	Special case: `(1/3,1/3,1/3)`
CCD	The original `mf.kpts`; `n` is the reference mesh shape	`1/(3*n_alpha)` on **all three axes**, including one-point axes

For example, on a `(1,1,4)` base mesh, MP3 shifts by `(0,0,1/12)` and CCD by `(1/3,1/3,1/12)`. Thus using the same reference and `flag_submesh=False` does not always give the same three orbital meshes. CCD has no `nks_mp` argument. In either method the concatenated correlation mesh contains `3N` points.

GW target and shifted meshes

GW takes an explicit `nks_target=n` and constructs two meshes in `_make_meshes`:

```
Kt = anchor + U(n)
Ks = Kt + h
h_alpha = 1/(2*n_alpha) if n_alpha > 1 else 0
```

For `n=(1,1,1)`, the special case is `h=(1/2,1/2,1/2)`. Each mesh has `N` points. Every point must be unique, and the two meshes must be disjoint.

Constructor	Anchor and orbital source	Global orbital container
`KRGWAC_stagger_submesh`	Anchor is the reference point closest to Gamma after wrapping. Both meshes are matched to existing SCF points.	The full original SCF mesh, with target and shifted index arrays
`KRGWAC_stagger_extend`	Anchor is zero. `get_bands` evaluates the target and shifted orbitals at the fixed SCF density.	The concatenation of `Kt` and `Ks`, containing `2N` points

GW submesh mode checks membership directly. A `(6,6,6)` Gamma-centered reference can supply a `(3,3,3)` target and its half-shifted partner. In GW extension mode, KRKS uses its existing `get_bands` backend; KRHF temporarily uses FFTDF and `vcut_sph`. Both constructors build a fresh complete GDF object with `j_only=False` on their global orbital container.

The main GW transfer mesh is constructed explicitly as

```
Q = minimum_image(Kt - Ks[0])
```

It contains `N` points, is equivalent to `U(n)-h`, and excludes Gamma. For each target point `k` and transfer `q`, `_load_lmn` finds the unique shifted point `k-q`. The negative and positive half-shifted transfer meshes are equivalent modulo the base grid.

Three-mesh MP3

Data flow and external amplitude

`KMP3_3mesh` binds `double_amplitude` to `mp3_amplitude_3mesh_efficient`. The older explicit-loop `mp3_amplitude_3mesh` remains available, but is not the default kernel path.

The constructor fixes the external configuration to

```
_mesh_mp3_amp = (0,0,1,2)
I,J on K0; A on K1; B on K2
```

`mesh_config_mp3` supplies the required auxiliary amplitudes and ERIs. With `v` denoting the stored ERI, including its `1/N` factor, the first-order amplitudes are

```
u_IJ^AB = conjugate(v_IJAB) / Delta_IJAB
Delta_IJAB = epsilon_I + epsilon_J - epsilon_A - epsilon_B
```

The five auxiliary configurations actually constructed are:

Amplitude meshes `(mi,mj,ma,mb)`	Use
`(0,0,1,2)`	External first-order amplitude and correction term
`(2,1,1,2)`	Occupied ladder, or 4h2p term
`(0,0,2,1)`	Virtual ladder, or 2h4p term
`(1,0,2,2)`	Mixed occupied–virtual contractions
`(2,0,1,1)`	Mixed occupied–virtual contractions

Each block has shape `(N,N,N,nocc,nocc,nvir,nvir)`; the fourth momentum is implicit. Only the external `(0,0,1,2)` combined MP3 amplitude is formed.

Meshes used in each contraction

The table gives every contribution to the second-order numerator `R_IJ^AB`. Each row includes sums over its occupied/virtual band indices and one independent internal momentum. The other internal momentum follows from conservation. `v_PQRS` uses the physicists' ordering `<PQ|RS>`.

Contribution to `R_IJ^AB`	Internal mesh assignment	Auxiliary amplitude meshes
`sum_KL v_KLIJ u_KL^AB`	`(K,L) = (2,1)`	`(2,1,1,2)`
`sum_CD v_ABCD u_IJ^CD`	`(C,D) = (2,1)`	`(0,0,2,1)`
`sum_KC (2*v_AKIC - v_AKCI) u_KJ^CB`	`(K,C) = (1,2)`	`(1,0,2,2)`
`-sum_KC v_AKIC u_KJ^BC`	`(K,C) = (1,2)`	`(1,0,2,2)`
`-sum_KC v_BKCI u_KJ^AC`	`(K,C) = (2,1)`	`(2,0,1,1)`
`sum_KC (2*v_BKJC - v_BKCJ) u_KI^CA`	`(K,C) = (2,1)`	`(2,0,1,1)`
`-sum_KC v_BKJC u_KI^AC`	`(K,C) = (2,1)`	`(2,0,1,1)`
`-sum_KC v_AKCJ u_KI^BC`	`(K,C) = (1,2)`	`(1,0,2,2)`

The last three rows are the simultaneous `(I,A) <-> (J,B)` partners of the preceding three mixed rows. In the first group, `-v_BKCI*u_KJ^AC` uses `(K,C)=(2,1)`, whereas the other mixed contractions use `(1,2)`. The equivalent `ovov` contractions in the source also use electron-interchange symmetry, such as `v_BKCI = v_KBIC`.

The selected ERI blocks are:

Tensor	Mesh tuples in that tensor's orbital-axis order
`oovv`	The five auxiliary amplitude configurations above
`oooo`	`(2,1,0,0)`
`vvvv`	`(1,2,2,1)`
`voov`	`(1,1,0,2)`, `(2,2,0,1)`
`ovov`	`(1,1,0,2)`, `(2,2,0,1)`

These lists are implemented in `mesh_config_mp3` and consumed by `_ERIS_3mesh`. Equal mesh tuples for `voov` and `ovov` still refer to different orbital spaces on their axes.

Correction and energy

With `madelung=True`, the efficient path evaluates a Gaussian-damped reciprocal-space correction on the one-third-shifted transfer grid using `madelung_stagger_adaptive`. If `D` is its integral-minus-quadrature value, the stored `madelung_constant` is `xi = -D`. The combined amplitude is

```
T_IJ^AB = u_IJ^AB + (R_IJ^AB + 2*xi*u_IJ^AB) / Delta_IJAB
```

This sign convention matches the MP3 outline's quadrature-minus-integral definition of `xi`. The correction defaults to **off**; staggering by itself does not activate it.

`mp3_energy_3mesh` contracts only the external configuration:

```
E[T] = (1/N) sum_IJAB (2*v_IJAB - v_IJBA) T_IJ^AB
```

The direct integral uses `(0,0,1,2)` and the exchange integral uses `(0,0,2,1)`. The inherited kernel evaluates `E2=E[u]`, `E2+E3=E[T]`, and their difference, stored as `e_corr_mp2`, `e_corr`, and `e_corr_mp3`, respectively. It returns `(e_corr, t2)`, where public `t2` is `u` and `t2_mp3` is the combined `T` when amplitude storage is enabled. The energy is in hartree per primitive cell.

The outline uses ERIs and amplitudes without the stored 1/N factors and writes an explicit 1/N^3 energy prefactor. The code's 1/N energy factor, 1/N in each ERI, and corresponding amplitude normalization give the same counting. N always means one submesh's size, not the global k-point count.

Three-mesh CCD

Coupled amplitude blocks and iteration

KRCCD_3mesh uses the PySCF CC iteration driver with custom amplitude update, initialization, energy, and intermediates. Singles remain zero. Unless initial amplitudes are supplied, doubles also start at zero; the first update gives the MP2 amplitudes when level_shift=0. mbpt2=True requests one update from the iteration driver.

The solver stores nine mesh configurations:

```
(0,0,1,2)  (0,1,1,0)  (0,1,2,2)
(0,2,1,1)  (0,2,2,0)  (1,1,0,2)
(1,2,0,0)  (1,2,2,1)  (2,2,0,1)
```

These represent the 18 configurations with mi != ma and momentum conservation, using T_IJ^AB = T_JI^BA. amp_3mesh retrieves a stored configuration or its simultaneous occupied/virtual exchange. The doubles array has shape (9,N,N,N,nocc,nocc,nvir,nvir).

Unlike MP3's restricted ERI list, _ERIS_3stagger allocates all 18 mesh configurations with mp != mr for each of oooo, oovv, ovov, voov, and vvvv. It builds integrals using momentum and ERI symmetry and divides each by N. The integrals and large intermediates use dense in-memory storage; unavailable out-of-core paths raise an error.

Meshes in the amplitude equation

For three distinct mesh labels x,y,z, the following table describes ccd_mesh_dependence_amp. It applies to all nine stored configurations and their symmetry partners.

Amplitude contribution	(I,J,A,B)=(x,x,y,z)	(I,J,A,B)=(y,z,x,x)	(I,J,A,B)=(x,y,y,x)
Bare v_ABIJ	External meshes only	External meshes only	External meshes only
sum_KL W_KLIJ T_KL^AB	(K,L)=(z,y)	(K,L)=(z,y)	(K,L)=(z,z)
sum_CD W_ABCD T_IJ^CD	(C,D)=(z,y)	(C,D)=(z,y)	(C,D)=(z,z)
sum_KC [(2*W_AKIC-W_AKCI) T_KJ^CB - W_AKIC T_KJ^BC]	(K,C)=(y,z)	(K,C)=(z,y)	(K,C)=(y,z)
-sum_KC W_BKCI T_KJ^AC	(K,C)=(z,y)	(K,C)=(z,y)	(K,C)=(z,z)

The last two rows are also evaluated after (I,A) <-> (J,B), by querying the dependency table for (mj,mi,mb,ma). These are the source's terms 10–12 and their partners. For (0,0,1,2), the outer contraction assignments are exactly the MP3 assignments above, now with CCD amplitudes and dressed intermediates.

For (x,y,y,x), the helper returns (y,z,z,x) for the combined mixed row, encoding two possible pairs. **The update takes only [:2], hence (y,z).** It does not average the alternative (z,x) route.

The remaining amplitude contributions are

```
+ sum_C Lvv_AC T_IJ^CB + sum_C Lvv_BC T_IJ^AC
- sum_K Loo_KI T_KJ^AB - sum_K Loo_KJ T_IK^AB
```

Here the contracted momentum is fixed: kC=kA or kB, and kK=kI or kJ, respectively. Thus Lvv uses the external virtual mesh and Loo the external occupied mesh. Their own internal mesh sums are specified below. The final numerator is divided by epsilon_I+epsilon_J-epsilon_A-epsilon_B, with level_shift added to each virtual energy. There is no separate off-diagonal Fock-matrix contraction in this CCD update.

Meshes inside the four-index intermediates

The four-index objects correspond to the outline's `chi_RS^PQ`:

Code intermediate	Definition or role
`Woooo`, `W_KLIJ`	`v_KLIJ + sum_CD v_KLCD T_IJ^CD`
`Wvvvv`, `W_ABCD`	Bare `v_ABCD`; there is no additional dressing
`Wvoov`, `W_AKIC`	Bare `v_AKIC` plus the two half-weighted `oovv*T` contractions
`Wvovo`, `W_AKCI`	Bare `v_AKCI` minus one half-weighted `oovv*T` contraction

`ccd_mesh_dependence_imb` takes mesh labels in **`(P,Q,R,S)` order**. Its assignments are:

Intermediate's internal sum	`chi_xx^yz`	`chi_yz^xx`	`chi_xy^yx`
`Woooo: (C,D)`	`(z,y)`	`(z,y)`	`(z,z)`
`Wvvvv`	No internal sum	No internal sum	No internal sum
`Wvoov: (L,D)`	`(z,y)`	`(y,z)`	Not built or needed
`Wvovo: (L,D)`	`(z,y)`	`(z,y)`	`(z,z)`

`Woooo` builds symmetry representatives and obtains their partners by transposition. `Wvoov` excludes `mK=mI` as well as `mA=mI`, which eliminates the last column for that intermediate. The bold entry differs from the printed CCD outline table; see the final comparison.

Meshes inside the two-index intermediates

The code computes the correlation parts of `kappa`, called `Loo` and `Lvv` in the update, on each mesh separately. For an external mesh `m`, write `p=(m+1)%3` and `r=(m+2)%3`.

Intermediate	Internal mesh triples, each with weight `1/4`
`Loo_KI`, with `kK=kI` on mesh `m`; tuple order `(L,C,D)`	`(m,p,r)`, `(m,r,p)`, `(r,p,p)`, `(p,r,r)`
`Lvv_AC`, with `kA=kC` on mesh `m`; tuple order `(K,L,D)`	`(p,r,m)`, `(r,p,m)`, `(p,p,r)`, `(r,r,p)`

For `Loo`, the contraction is `(2*v_KLCD-v_KLDC) T_IL^CD`. For `Lvv`, it is `-(2*v_KLCD-v_KLDC) T_KL^AD`. `cc_Foo_3mesh` and `cc_Fvv_3mesh` sum all four configurations and divide by four. This is an explicit averaging choice.

Correction and energy

CCD defaults to `madelung=False`. When enabled, it precomputes integral-minus-quadrature constants `D[0]`, `D[1]`, and `D[2]` for transfer shifts `0`, `delta`, and `2*delta`, using `madelung_stagger`. The amplitude numerator receives these multiples of its current `T`:

Contraction group	Added coefficient, with mesh differences modulo three
Occupied ladder	`+D[mi-mK]`, with `mK` from the ladder assignment
Virtual ladder	`+D[mC-ma]`, with `mC` from the ladder assignment
Combined mixed row	`-D[mC-ma]`
Last mixed row	`-D[mC-mb]`
Partners of the two mixed rows	The same rules after swapping `(I,A)` with `(J,B)`

These corrections are applied in `update_amps_3mesh`; they are not extra terms in the final energy contraction. Their evaluation uses the explicit reciprocal grid, rather than MP3's adaptive evaluator.

`energy_3mesh` averages six energy evaluations, each using `(1/N) sum (2*v_IJAB-v_IJBA) T_IJ^AB`:

```
(0,0,1,2), (1,2,0,0), (1,1,0,2),
(0,2,1,1), (2,2,0,1), (0,1,2,2)
```

These are the energy-safe configurations and their distinct symmetry representatives. The three stored exchange-type configurations `(0,1,1,0)`, `(0,2,2,0)`, and `(1,2,2,1)` participate in the coupled amplitude equations but are excluded from the energy average. For them, `I` and `B` share a mesh, so the exchange ERI in the energy would sample zero transfer. `kernel()` returns `(e_corr,t1,t2)`; the correlation energy is in hartree per primitive cell.

Staggered G0W0 with analytic continuation

Meshes used by each quantity

Both public constructors use the shared `KRGWAC_stagger` kernel. Target states are on `Kt`; internal propagation is on `Ks`. The response combines both meshes, rather than being calculated solely on `Ks`.

Quantity	External or left states	Summed or right states	Transfer mesh
Reference potential `v_mf`	Target MOs	Original SCF density on the full reference mesh	Evaluated at target band points
Fock exchange `Sigma_x`	Target MOs	Occupied shifted MOs and their density matrix	Differences between target and shifted points
Insulating response `Pi(q,i*nu)`	Occupied target states	Virtual shifted states at `k-q`	`Q=U(n)-h`
Screened correlation interaction `Wc(q,i*nu)`	Auxiliary DF space	Response built from target-to-shifted transitions	The same `Q`
Main diagonal correlation self-energy	Requested target orbital `n,k`	All shifted bands `m,k-q`, occupied and virtual	The same `Q`
Long-wavelength response body	Occupied target states	Virtual target states at the same `k`	A separate `q=0` calculation
Long-wavelength dipoles, head, and wings	Target occupied states	Target virtual states at the same `k`	Small-`q` limit evaluated from target data
Green-function factor in the correlation correction	Requested target state	Its own reference energy	The `q -> 0`, same-state limit

`_get_v_mf` computes the reference exchange-correlation contribution as `get_veff-get_j` and projects it onto target orbitals. `_get_sigma_exchange` instead constructs a KRHF object on `Ks` with `exxdiv=None`, using the shifted density and evaluating exchange at selected target points. The fresh GDF object on `gw` supplies the response and correlation DF tensors; the exchange helper uses its separately constructed KRHF object.

Response and main self-energy

For each `q`, `_load_lmn` transforms GDF three-center integrals into `L_Pnm(k,k-q)`. Its matching condition is `-k+(k-q)+q=G`; `right_index` records the shifted partner of every target point.

For integer closed-shell occupations, `_rho_response_stagger` evaluates

```
Pi_PQ(q,i*nu) = (4/N) sum_kia
    L_Pia(k,k-q) conjugate(L_Qia(k,k-q))
    * (epsilon_i,k - epsilon_a,k-q)
    / (nu^2 + (epsilon_i,k - epsilon_a,k-q)^2)

Wc = inverse(I - Pi) - I
```

The Coulomb metric is incorporated in the DF factors. The code contracts `L* Wc L`, sums over all shifted propagation bands, and integrates the Green-function factor

```
(z - epsilon_m,k-q) / ((z - epsilon_m,k-q)^2 + nu^2)
```

with the negative quadrature weight `-w_nu/(pi*N)`. `_get_sigma_diag_stagger` performs this calculation for all `q` in `Q`. Selecting `kptlist` or `orbs` reduces the requested self-energy outputs; the screening response still uses the full target and shifted meshes and the full occupied/virtual spaces.

Positive imaginary integration frequencies and weights come from scaled Gauss–Legendre quadrature (`nw=100` by default). Self-energy samples lie at `z=ef+i*omega`, including `omega=0`; `ac_iw_cutoff=5.0` hartree by default limits the samples used for continuation, not the screening integration grid. For an insulator, `ef` is the midpoint between the highest occupied and lowest virtual energies in the global orbital container.

The fractional-occupation branch sums all target/shifted band pairs with occupation differences and a `2/N` prefactor. It requires an explicit `gw.ef`, uses at least 200 integration frequencies, and skips the long-wavelength correlation correction. The primary insulating workflow above is the one corresponding to the outline's occupied–virtual formulas.

Exchange and correlation corrections

`fc`, `fc_ex_madelung`, and `fc_corr_madelung` all default to `True`. `fc=False` disables both corrections; the other two flags select them independently.

For exchange, `_madelung_stagger` evaluates the integral-minus-quadrature constant `D_h` on the actual half-shifted transfer grid. The code subtracts `D_h` from the occupied target diagonal entries of `Sigma_x`.

The correlation correction in `_correlation_finite_size_correction` uses additional **target-to-target** DF blocks at `q=0`. For each integration frequency it constructs:

1. The response body `B = I-Pi_target`, using occupied and virtual target orbitals at equal k-points.
2. Target dipoles from `_get_rmat`, using level-4 Becke integration of AO gradients and the ratio `<psi_i|nabla|psi_a>/(epsilon_a-epsilon_i)`.
3. The head tensor `F`, wing tensor `U`, and dielectric tensor `L = F-U^dagger B^(-1) U`, with contractions in the code's auxiliary basis.

The code requires this `L` to be real within tolerance and positive definite. It transforms the lattice by `L^(-1/2)` and evaluates another shifted Madelung quadrature on that transformed cell, preserving the fractional mesh shift. Defining

```
xi_I = volume_original * D_h(original cell) / (4*pi)
xi_L = volume_transformed * D_h(transformed cell) / (4*pi)
xi   = xi_L - xi_I
```

the correction to a target state's self-energy sample is accumulated as

```
delta_Sigma_c(z) = -sum_nu [4*xi(nu)/volume_original] * w_nu
    * (z-epsilon_n,k) / ((z-epsilon_n,k)^2 + nu^2)
```

This is a separate singularity-subtraction calculation. It does not add `q=0` to the main staggered transfer sum.

Analytic continuation and quasiparticle energies

The sampled correlation self-energy is fitted with PySCF `PadeAC` or `TwoPoleAC` using absolute frequencies `ef+i*omega`. For each selected target state, the quasiparticle equation is

```
E = epsilon_ref + Re[Sigma_c(E) + Sigma_x - v_mf]
```

The default is a nonlinear scalar root solve. `qpe_linearized=True` instead uses a derivative-based renormalization factor; by default, factors outside `[0.5,1.5]` are replaced by one. Either option remains one-shot G0W0: the reference orbitals, energies, and screening are held fixed.

`kernel()` returns `None` and writes target-grid results to `mo_energy`, with shape `(N,nmo)`. Unrequested or failed entries remain `NaN`; `qpe_converged` records statewise success. The implementation supports restricted three-dimensional cells, diagonal self-energies, and no frozen orbitals.

Differences from the method outlines

MP3 compared with the September outline

The mesh assignments implement the outline's three-mesh construction and its `(I,J,A,B)=(x,x,y,z)` energy formula, with the following concrete choices.

Topic	Outline	Current implementation
External energy sampling	Sections 3.1–3.2 allow permutations of `(x,x,y,z)` and the alternative `(y,z,x,x)`.	The constructor fixes `(0,0,1,2)` and does not average external configurations. The helper contains another configuration family, but it is not exposed as a constructor choice.
Mixed-term grouping	Equation (3.3) groups `-v_AKCJ*u_KI^BC` with the other A-side terms.	The implementation groups its permutation partner `-v_BKCI*u_KJ^AC` with the A-side terms. After including both permutation groups, these are the same contractions and mesh assignments.
Madelung correction	Section 3.3 includes the correction in the final proposed method.	It is optional and defaults off. When enabled, the efficient path uses the combined `2*xi*u/Delta` formula, with an adaptive Gaussian reciprocal quadrature.
Reference and mesh construction	Section 2 assumes thermodynamic-limit orbitals; Section 3 shifts an arbitrary uniform mesh in periodic directions.	Orbitals come from a finite SCF calculation or fixed-density band extension. Extension normally leaves one-point axes unshifted, except for `(1,1,1)`.
Amplitude and normalization	The outline's MP3 amplitude includes its MP2 part, with explicit mesh factors in the equations.	That combined amplitude is `t2_mp3`; public `t2` is first order. Stored ERIs absorb `1/N`. These naming and normalization differences do not change the corresponding energy formula.

CCD compared with the three-mesh outline

The implementation follows the outline's coupled amplitude forms and intermediate decomposition. It also makes choices where the outline leaves the sampling unresolved, and differs from one printed intermediate entry.

Topic	Outline	Current implementation
Exchange-type amplitude `(x,y,y,x)`	Section 0.3 permits `(K,C)=(y,z)` or `(z,x)` for the combined mixed terms and marks the choice unresolved.	`update_amps_3mesh` uses only `(y,z)`. The alternative pair is returned by the helper but discarded by `[:2]`.

Continued on next page

Topic	Outline	Current implementation
Two-index intermediates	Section 0.5 lists four admissible configurations and leaves the practical choice unresolved.	`Loo` and `Lvv` average all four with equal weights. Their spin-adapted contractions use `2*direct-exchange`, including the virtual minus sign, as in Section 0.2; the sums printed in Section 0.5 are not the literal implemented coefficients.
`Wvoov` with `chi_yz^xx`	The Section 0.4 table on page 4 prints `(L,D)=(z,y)`.	`ccd_mesh_dependence_imb` uses `(y,z)`. For example, `chi_12^00` uses `(L,D)=(1,2)`. This satisfies that section's stated constraints `D != I` and `D != K`; the printed `(2,1)` would put `D` on `I`'s mesh. Thus the table and its preceding constraints disagree, and the code follows the constraints for this entry.
Energy and symmetry	Section 0.1 lists 12 admissible energy samplings, with symmetry discussed earlier.	Nine amplitude representatives are stored, and the energy averages six representatives of the two admissible energy families. The exchange-type amplitudes are retained only for the coupled equations.
Madelung and mesh options	The CCD outline does not specify an additional Madelung implementation or SCF/band-extension workflow.	Optional per-contraction corrections are present and default off. Band extension shifts all three axes, including one-point axes; submesh mode shifts only axes sampled more than once in the reference.

The existing validation notes report that this symmetry-reduced CCD implementation agrees with the archived symmetry-reduced solver but differs from the archived `nosymm` result by about `1.92e-6` Ha on its small H2 example. Those notes identify an additional averaged exchange-type contraction route in `nosymm`. Selecting a route in the outline's unresolved case therefore has a documented numerical consequence; the present description does not establish which formulation is preferable.

G0W0 compared with the September outline

The main calculation follows the half-shifted transfer quadrature of Equation (8), with a separate anisotropic singularity correction. The most substantial approximation relative to the later appendix is the treatment of the small-momentum transition densities.

Topic	Outline	Current implementation
Representation and output	Sections 1–2 give a plane-wave expression for general `Sigma_nn'`.	Screening and self-energy are contracted in a Gaussian DF auxiliary basis, only diagonal entries are computed, and analytic continuation plus scalar quasiparticle equations produces real-frequency orbital energies.
Half shifts and orbital data	Section 2 prescribes a half-grid transfer shift and assumes accurate response and frequency integrals in its error discussion.	The constructor specifies finite target and shifted orbital meshes. One-point axes are normally unshifted, with the all-one mesh as an exception. Screening uses target occupied and shifted virtual states; its discretization is part of the calculation.
Small-q derivatives	Appendix .2.1 gives the gradient approximation and warns about crystalline Gaussian bases. Appendix .2.2 introduces Hamiltonian, overlap, and basis-derivative terms.	`_get_rmat` implements the .2.1 gradient approximation. The additional `.2.2` terms are absent. This affects the target-mesh dielectric tensor and correlation correction.
Printed head prefactor	Appendix .2.1 on page 7 prints `8*pi/(N*volume)` in the head tensor `F`.	`_get_fab` uses `16*pi/(N*volume)` with the same occupied–virtual energy factor and dipole product. The latter follows from the `4/(N*volume)` closed-shell response prefactor printed earlier in the note and the second derivative of the quadratic head. This is a discrepancy within the printed formulas, not by itself evidence that the code's factor is wrong.

Continued on next page

Topic	Outline	Current implementation
Correction scope	Section 2 describes correlation singularity subtraction for staggered meshes and also gives a corrected standard-mesh formula, Equation (9).	The staggered classes have independent exchange and correlation Madelung flags, enabled by default. They do not implement Equation (9) as a separate standard-mesh solver. The correlation correction uses target-only `q=0` data and is skipped for fractional occupations.

The validation notes also report a finite-q overlap check that disagrees with the implemented gradient dipole on the H2 example. Agreement of the main response contractions or selected quasiparticle energies therefore does not independently validate the small-q correction required by the outline.